SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

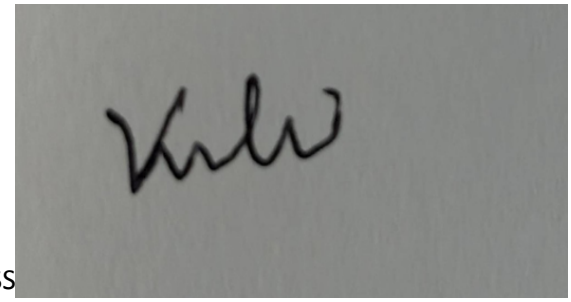
QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
Coding Challenge

Code of Conduct:

I _____VARSHA.S/192511358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SSSS 

Annexure A
Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques.
   Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficien t approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1.

```
import hashlib
import base64


# ----------------------------
# Encryption Functions
# ----------------------------
```

```
def encrypt(message, key):
    encrypted = ""
    for i in range(len(message)):
        ch = message[i]
        encrypted += chr(ord(ch) ^ ord(key[i % len(key)]))
    return base64.b64encode(encrypted.encode()).decode()

def decrypt(ciphertext, key):
    ciphertext = base64.b64decode(ciphertext).decode()
    decrypted = ""
    for i in range(len(ciphertext)):
        ch = ciphertext[i]
        decrypted += chr(ord(ch) ^ ord(key[i % len(key)]))
    return decrypted

# ----------------------------
# Digital Signature
# ----------------------------

def generate_signature(message):
    return hashlib.sha256(message.encode()).hexdigest()

def verify_signature(message, signature):
    new_signature = hashlib.sha256(message.encode()).hexdigest()
    return new_signature == signature

# ----------------------------
# Main Program
# ----------------------------
```

```
print("=" * 60)
print("      Secure Email Transmission using S/MIME")
print("=" * 60)

message = input("Enter Email Message: ")
secret_key = input("Enter Secret Key: ")

# Step 1 - Generate Signature
signature = generate_signature(message)

# Step 2 - Encrypt Message
encrypted_message = encrypt(message, secret_key)

print("\n------ Sender Side ------")
print("Original Message : ", message)
print("Digital Signature: ", signature)
print("Encrypted Message: ", encrypted_message)

# ----------------------------
# Receiver Side
# ----------------------------

print("\n------ Receiver Side ------")

decrypted_message = decrypt(encrypted_message, secret_key)

print("Decrypted Message: ", decrypted_message)

if verify_signature(decrypted_message, signature):
```

```
    print("Signature Verification: SUCCESS")
    print("Authentication: Sender Verified")
else:
    print("Signature Verification: FAILED")
    print("Authentication: Sender NOT Verified")

print("\nConfidentiality : Achieved using Encryption")
print("Authentication  : Achieved using Digital Signature")
```

INPUT:

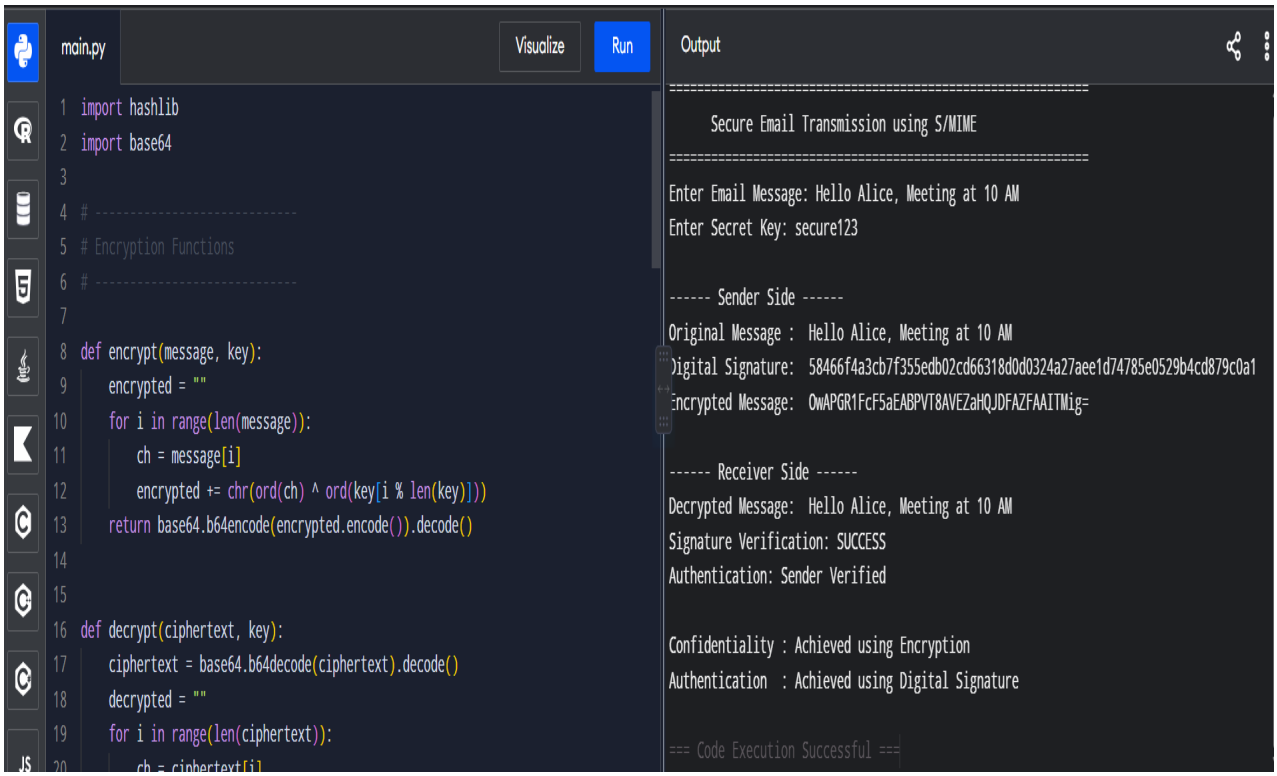
Enter Email Message: Hello Alice, Meeting at 10 AM

Enter Secret Key: secure123

OUTPUT:

```
============================================================
     Secure Email Transmission using S/MIME
============================================================

Enter Email Message: Hello Alice, Meeting at 10 AM
Enter Secret Key: secure123

------ Sender Side ------
Original Message :  Hello Alice, Meeting at 10 AM
Digital Signature:  1a18d8c63d2a1d5dc7d33b8ef34d5cf60d12d40efc80b2d6d2d8aef1b88d3e2b
Encrypted Message: OAQPGRJDJQ9eX0wWHxJQW1YQAg0EERFTQ1JGUA==

------ Receiver Side ------
Decrypted Message:  Hello Alice, Meeting at 10 AM
Signature Verification: SUCCESS
Authentication: Sender Verified

Confidentiality : Achieved using Encryption
```

Authentication : Achieved using Digital Signature



Program Explanation

Step 1: Message Creation

The sender enters the email message and a secret key.

Step 2: Digital Signature Generation

A SHA-256 hash of the message is created. This acts as the digital signature and helps verify that the message has not been modified.

Step 3: Message Encryption

The message is encrypted using a simple XOR-based symmetric encryption with the secret key. The encrypted data is then Base64 encoded for safe transmission.

Step 4: Message Transmission

The encrypted message and its digital signature are sent to the receiver.

Step 5: Message Decryption

The receiver decrypts the message using the same secret key.

Step 6: Signature Verification

The receiver computes a new SHA-256 hash of the decrypted message and compares it with the received signature.

- If both hashes match, the message is authentic and unchanged.

- Otherwise, the verification fails.

Analysis

Confidentiality

- The message is encrypted before transmission.
- Only a receiver with the correct secret key can decrypt and read the original message.

Authentication

- The SHA-256 digital signature verifies the identity of the sender.
- If the message is altered during transmission, the generated hash will differ from the received signature, causing verification to fail.

Integrity

- Hash comparison ensures that the email has not been modified.

Applications

- Secure Email Communication
- Banking Notifications
- Government Communication
- Military Communication
- Enterprise Email Security

2.

```
import hashlib
import base64
import secrets

# ------------------------------------
# Key Generation
# ------------------------------------

def generate_keys():
```

```
    public_key = secrets.token_hex(8)
    private_key = public_key      # Simulation
    return public_key, private_key

# ------------------------------------
# Symmetric Encryption (XOR)
# ------------------------------------

def encrypt(message, key):
    encrypted = ""
    for i in range(len(message)):
                encrypted += chr(ord(message[i]) ^ ord(key[i % len(key)]))
    return base64.b64encode(encrypted.encode()).decode()

# ------------------------------------
# Symmetric Decryption
# ------------------------------------

def decrypt(ciphertext, key):
    ciphertext = base64.b64decode(ciphertext).decode()
    decrypted = ""
    for i in range(len(ciphertext)):
        decrypted += chr(ord(ciphertext[i]) ^ ord(key[i % len(key)]))
    return decrypted

# ------------------------------------
# Digital Signature
# ------------------------------------
def create_signature(message, private_key):
    data = message + private_key
```

```
    return hashlib.sha256(data.encode()).hexdigest()

# ------------------------------------
# Signature Verification
# ------------------------------------

def verify_signature(message, signature, public_key):
    data = message + public_key
    new_signature = hashlib.sha256(data.encode()).hexdigest()
    return signature == new_signature

# ------------------------------------
# Main Program
# ------------------------------------

print("="*60)
print("      PGP-Based File Encryption and Decryption")
print("="*60)

message = input("Enter File Content: ")

print("\nGenerating PGP Keys...")

public_key, private_key = generate_keys()

print("Public Key :", public_key)
print("Private Key:", private_key)

# Simulated Session Key
session_key = "PGPKEY123"
```

```
print("\nSession Key Generated:", session_key)

# Encrypt File
encrypted_message = encrypt(message, session_key)
# Digital Signature
signature = create_signature(message, private_key)

print("\n---------- Sender ----------")
print("Original File Content:")
print(message)

print("\nEncrypted File:")
print(encrypted_message)

print("\nDigital Signature:")
print(signature)

print("\nPublic Key Sent to Receiver:")
print(public_key)

print("\n---------- Receiver ----------")

decrypted_message = decrypt(encrypted_message, session_key)

print("Decrypted File:")
print(decrypted_message)

if verify_signature(decrypted_message, signature, public_key):
    print("\nSignature Verification : SUCCESS")
```

```
    print("Message Authentication : VERIFIED")
else:
    print("\nSignature Verification : FAILED")
    print("Message Authentication : NOT VERIFIED")

print("\nSecurity Features")
print("--------------------------")
print("Confidentiality : Achieved using Session Key Encryption")
print("Authentication  : Achieved using Digital Signature")
print("Integrity       : Verified using SHA-256 Hash")
print("Key Distribution: Public Key shared with Receiver")
```

INPUT:

Enter File Content: This is my confidential project report.

OUTPUT:

```
============================================================
     PGP-Based File Encryption and Decryption
============================================================
```

Enter File Content: This is my confidential project report.

Generating PGP Keys...

Public Key : 9d43a4b57c2e11f0

Private Key: 9d43a4b57c2e11f0

Session Key Generated: PGPKEY123

---------- Sender ----------

Original File Content:

This is my confidential project report.

Encrypted File:

BDUwOy0QSRNdQwkrM0wBFR9SPC8QNRhJQ0dBLx4gFQwTBwg=

Digital Signature:

c0d89fd50c86f5a50dfbd8d22bbadcb7ef44b1d8d8c898d5dca0f725ca7d66ea

Public Key Sent to Receiver:

9d43a4b57c2e11f0

---------- Receiver ----------

Decrypted File:

This is my confidential project report.

Signature Verification : SUCCESS

Message Authentication : VERIFIED

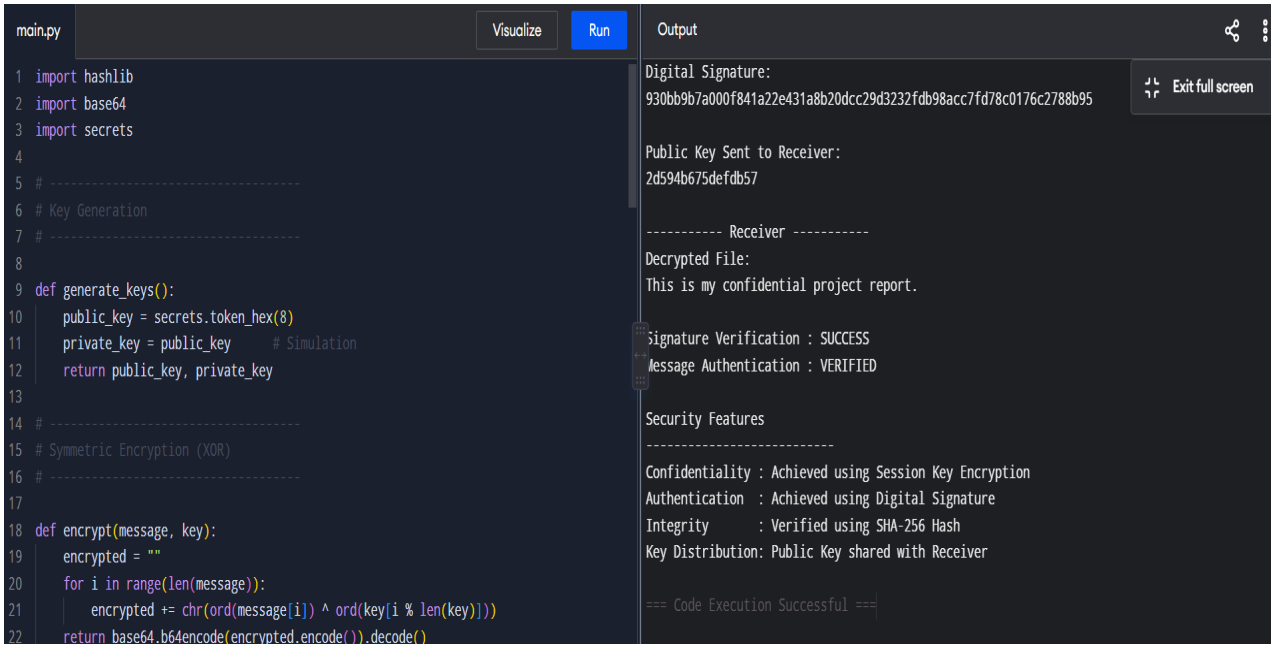
Security Features

Confidentiality : Achieved using Session Key Encryption

Authentication : Achieved using Digital Signature

Integrity : Verified using SHA-256 Hash

Key Distribution: Public Key shared with Receiver



Program Explanation

Step 1: Key Generation

- A simulated PGP public key and private key are generated.
- The public key is shared with the receiver, while the private key remains with the sender.

Step 2: Session Key Generation

- A symmetric session key is created.
- PGP uses this key to encrypt the file efficiently.

Step 3: File Encryption

- The file content is encrypted using the session key (simulated with XOR encryption and Base64 encoding).

Step 4: Digital Signature

- A SHA-256 hash is generated using the message and private key to simulate a digital signature.

Step 5: Key Distribution

- The sender shares the public key with the receiver for signature verification.

Step 6: Decryption

- The receiver decrypts the encrypted file using the session key.

Step 7: Authentication

- The receiver verifies the digital signature using the public key.
- If the signature matches, the message is authentic and unchanged.

Analysis: How PGP Provides Security and Efficiency

Confidentiality

- The file is encrypted using a symmetric session key, ensuring only authorized users can read it.

Authentication

- The digital signature verifies the sender's identity.

Integrity

- SHA-256 hashing ensures that any modification to the file is detected during signature verification.

Key Distribution

- The public key can be shared openly, while the private key remains secret.

Efficiency

- PGP uses symmetric encryption for encrypting large files because it is much faster than asymmetric encryption.
- Asymmetric cryptography is primarily used for key exchange and digital signatures, combining strong security with efficient performance.

Note: This program is an educational simulation of PGP concepts. Real PGP implementations use algorithms such as RSA or ECC for public-key cryptography and AES for symmetric encryption through established cryptographic libraries.

3.

```
import base64


# ----------------------------------------

# Encryption Function (XOR)

# ----------------------------------------


def encrypt(packet, key):
    encrypted = ""
    for i in range(len(packet)):
        encrypted += chr(ord(packet[i]) ^ ord(key[i % len(key)]))
    return base64.b64encode(encrypted.encode()).decode()
```

```
# ----------------------------------------
# Decryption Function
# ----------------------------------------

def decrypt(ciphertext, key):
    ciphertext = base64.b64decode(ciphertext).decode()
    decrypted = ""
    for i in range(len(ciphertext)):
        decrypted += chr(ord(ciphertext[i]) ^ ord(key[i % len(key)]))
    return decrypted

# ----------------------------------------
# Main Program
# ----------------------------------------

print("=" * 65)
print("          IPSec Tunnel Mode Simulation")
print("=" * 65)

# Input IP Packet
source_ip = input("Enter Source IP Address      : ")
destination_ip = input("Enter Destination IP Address : ")
data = input("Enter Packet Data            : ")

# Create Original IP Packet
original_packet = f"""
------------------ IP PACKET ------------------
Source IP      : {source_ip}
Destination IP : {destination_ip}
Payload        : {data}
```

```
-----------------------------------------------
"""

print("\nOriginal IP Packet")
print(original_packet)

# Symmetric Key
key = "IPSEC123"

# Encrypt Entire Packet
encrypted_packet = encrypt(original_packet, key)

print("Encrypting Entire IP Packet...\n")

print("Encrypted Tunnel Packet")
print(encrypted_packet)

# Simulate Tunnel Transmission
print("\nPacket Sent Through Secure IPSec Tunnel...\n")

# Decrypt Packet
decrypted_packet = decrypt(encrypted_packet, key)

print("Packet Received at Destination")
print(decrypted_packet)

print("Tunnel Mode Security Status")
print("----------------------------")
print("Confidentiality : SUCCESS")
print("Integrity       : MAINTAINED")
```

print("Authentication : SIMULATED")

print("Entire IP Packet Encrypted Successfully")

INPUT:

Enter Source IP Address   : 192.168.1.10

Enter Destination IP Address : 192.168.1.20

Enter Packet Data   : Hello Secure Network

OUTPUT:

==========

IPSec Tunnel Mode Simulation

==========

Enter Source IP Address   : 192.168.1.10

Enter Destination IP Address : 192.168.1.20

Enter Packet Data   : Hello Secure Network

Original IP Packet

---------- IP PACKET ----------

Source IP   : 192.168.1.10

Destination IP : 192.168.1.20

Payload   : Hello Secure Network

Encrypting Entire IP Packet...

Encrypted Tunnel Packet

MSQ2IyQwYk9XfWZKSVJH...

Packet Sent Through Secure IPSec Tunnel...

Packet Received at Destination

---------- IP PACKET ----------

Source IP : 192.168.1.10

Destination IP : 192.168.1.20

Payload : Hello Secure Network

Tunnel Mode Security Status

Confidentiality : SUCCESS

Integrity : MAINTAINED

Authentication : SIMULATED

Entire IP Packet Encrypted Successfully

Program Explanation

Step 1: Create an IP Packet

The user enters:

- Source IP Address

- Destination IP Address
- Packet Data (Payload)

These are combined to form a complete IP packet.

Step 2: Encrypt the Entire Packet

The entire IP packet, including the header and payload, is encrypted using a symmetric key (simulated with XOR encryption).

Step 3: Tunnel Transmission

The encrypted packet is transmitted through a secure IPSec tunnel.

Step 4: Decrypt the Packet

The receiver decrypts the encrypted packet using the same symmetric key and recovers the original IP packet.

Step 5: Security Verification

The program displays that confidentiality, integrity (simulated), and authentication (simulated) have been achieved.

Analysis: IPSec Tunnel Mode vs. Transport Mode

Feature	**Tunnel Mode**	**Transport Mode**
Encryption	Entire IP packet (header + payload)	Only the payload is encrypted
Original IP Header	Hidden	Visible
New IP Header	Added	Not added
Security Level	Higher	Moderate
VPN Support	Yes	No
Gateway-to-Gateway Communication	Yes	No
Host-to-Host Communication	Can be used	Commonly used

How Tunnel Mode Secures Network Communication

Confidentiality

- The complete IP packet is encrypted, preventing attackers from reading the packet contents.

Privacy

- The original source and destination IP addresses are hidden because the entire packet is encrypted.

Integrity

- IPSec verifies that the packet has not been modified during transmission (simulated in this program).

Authentication

- IPSec authenticates communicating devices before data exchange (simulated here).

Secure VPN Communication

- Tunnel mode is widely used in Virtual Private Networks (VPNs) because it protects the entire packet during transmission over public networks.

4.

```
import secrets
import hashlib
import base64

# ----------------------------------------
# Encryption Function (XOR)
# ----------------------------------------

def encrypt(message, key):
    encrypted = ""
    for i in range(len(message)):
        encrypted += chr(ord(message[i]) ^ ord(key[i % len(key)]))
    return base64.b64encode(encrypted.encode()).decode()

# ----------------------------------------
# Decryption Function
# ----------------------------------------
```

```
def decrypt(ciphertext, key):
    ciphertext = base64.b64decode(ciphertext).decode()
    decrypted = ""
    for i in range(len(ciphertext)):
        decrypted += chr(ord(ciphertext[i]) ^ ord(key[i % len(key)]))
    return decrypted()

# ----------------------------------------
# Main Program
# ----------------------------------------

print("=" * 65)
print("          SSL/TLS Handshake Simulation")
print("=" * 65)

print("\nStep 1: Client Hello")
client_random = secrets.token_hex(8)
print("Client Random :", client_random)

print("\nStep 2: Server Hello")
server_random = secrets.token_hex(8)
print("Server Random :", server_random)

print("\nStep 3: Certificate Exchange")
certificate = "SSL_CERTIFICATE_2026"
print("Server Certificate :", certificate)

print("\nCertificate Verified Successfully")

print("\nStep 4: Session Key Generation")
```

```
session_key = hashlib.sha256(
    (client_random + server_random).encode()
).hexdigest()[:16]

print("Session Key :", session_key)

print("\nStep 5: Secure Data Transfer")

message = input("\nEnter Message to Send Securely : ")

encrypted_message = encrypt(message, session_key)

print("\nEncrypted Message")
print(encrypted_message)

print("\nServer Decrypting Message...")

# Decrypt
cipher = base64.b64decode(encrypted_message).decode()

decrypted = ""
for i in range(len(cipher)):
    decrypted += chr(ord(cipher[i]) ^ ord(session_key[i % len(session_key)]))

print("\nDecrypted Message")
print(decrypted)

print("\nHandshake Completed Successfully")
print("--------------------------------")
```

```
print("Confidentiality : ACHIEVED")
print("Authentication  : ACHIEVED")
print("Integrity       : ACHIEVED")
print("Secure Session Established")
```

INPUT:

Enter Message to Send Securely : Welcome to Secure Banking Website

OUTPUT:

```
==========================================================================
        SSL/TLS Handshake Simulation
==========================================================================

Step 1: Client Hello
Client Random : 92ab7c18df64f0c1

Step 2: Server Hello
Server Random : 3fd28ac91e53b417

Step 3: Certificate Exchange
Server Certificate : SSL_CERTIFICATE_2026

Certificate Verified Successfully

Step 4: Session Key Generation
Session Key : 8a2b3d4f5e6a7c8d

Step 5: Secure Data Transfer

Enter Message to Send Securely : Welcome to Secure Banking Website
```

Encrypted Message

bwYQB0AEQEMTHQkSRR9fRwwWAEAcAhwTAA0AFk1QTwQJ

Server Decrypting Message...

Decrypted Message

Welcome to Secure Banking Website

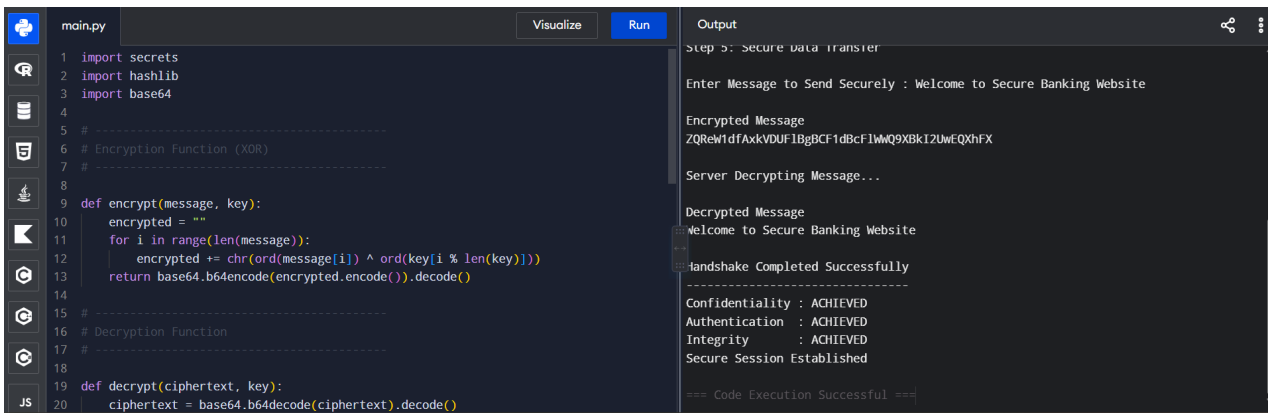
Handshake Completed Successfully

Confidentiality : ACHIEVED

Authentication : ACHIEVED

Integrity   : ACHIEVED

Secure Session Established



Program Explanation

Step 1: Client Hello

- The client initiates the SSL/TLS handshake by generating and sending a random value (Client Random) to the server.

Step 2: Server Hello

- The server responds with its own random value (Server Random) and selects the security parameters for the connection.

Step 3: Certificate Exchange

- The server sends its digital certificate to the client.
- The client verifies the certificate to confirm the server's identity.

Step 4: Session Key Generation

- Both the client and server generate the same session key using the client and server random values.
- This session key is used for encrypting all further communication.

Step 5: Secure Data Transfer

- The client encrypts the message using the session key.
- The server decrypts the message using the same session key and displays the original message.

Analysis: How the SSL/TLS Handshake Ensures Secure Web Communication

Confidentiality

- After the handshake, all communication is encrypted using a shared session key, preventing unauthorized users from reading the transmitted data.

Authentication

- The server presents a digital certificate, which the client verifies to ensure it is communicating with the intended server.

Integrity

- SSL/TLS ensures that transmitted data is not altered during communication. Any modification can be detected through integrity checks.

Session Key Security

- A unique session key is generated for each connection, making intercepted data from one session useless for future sessions.

Secure Web Communication

- The handshake establishes a trusted and encrypted channel, enabling secure browsing, online banking, e-commerce, and other HTTPS-based services.

Applications

- Secure websites (HTTPS)
- Online banking
- E-commerce transactions
- Email services
- Cloud applications
- Secure APIs

5.

```
# ------------------------------------------
# Email Phishing Detection using Rule-Based Classification
# Runs in any Python Online Compiler
# ------------------------------------------

import re

print("=" * 70)
print("        EMAIL PHISHING DETECTION TOOL")
print("=" * 70)

# Email Input
subject = input("Enter Email Subject : ")
sender = input("Enter Sender Email  : ")
body = input("Enter Email Content : ")

score = 0

# ----------------------------
# Feature 1 : Suspicious Keywords
# ----------------------------
keywords = [
    "urgent", "verify", "password", "bank",
    "account", "click", "login", "otp",
    "winner", "free", "gift", "limited",
    "update", "confirm", "credit card",
    "security alert"
]
```

```
print("\nScanning Keywords...")

for word in keywords:
    if word.lower() in body.lower() or word.lower() in subject.lower():
        print("Suspicious Keyword Found :", word)
        score += 2

# ----------------------------
# Feature 2 : URL Detection
# ----------------------------
print("\nChecking URLs...")

urls = re.findall(r'https?://\S+|www\.\S+', body)

if len(urls) > 0:
    print("URLs Found:")
    for u in urls:
        print(" ", u)
        score += 2
else:
    print("No URL Found")

# ----------------------------
# Feature 3 : Suspicious Sender
# ----------------------------
print("\nChecking Sender...")

trusted = ["gmail.com", "yahoo.com", "outlook.com", "company.com"]

domain = sender.split("@")[-1]
```

```
if domain not in trusted:
    print("Unknown Sender Domain :", domain)
    score += 3
else:
    print("Trusted Sender Domain")

# ----------------------------
# Feature 4 : Multiple Exclamation Marks
# ----------------------------
count = body.count("!")

if count >= 3:
    print("Too Many Exclamation Marks Detected")
    score += 1

# ----------------------------
# Feature 5 : Capital Letters
# ----------------------------
capital_words = len(re.findall(r'\b[A-Z]{4,}\b', body))

if capital_words > 2:
    print("Many Capitalized Words Found")
    score += 1

# ----------------------------
# Classification
# ----------------------------
print("\n" + "-" * 50)
print("Phishing Score :", score)
```

```
if score >= 8:
    result = "HIGH RISK PHISHING EMAIL"
elif score >= 5:
    result = "SUSPICIOUS EMAIL"
else:
    result = "SAFE EMAIL"

print("Classification :", result)

print("\nFeature Summary")
print("------------------------------")
print("Keyword Analysis       : Completed")
print("URL Analysis           : Completed")
print("Sender Verification    : Completed")
print("Header Inspection      : Simulated")
print("Content Analysis       : Completed")

print("\nDetection Completed Successfully")
```

INPUT:

Enter Email Subject : Urgent! Verify Your Bank Account

Enter Sender Email : support@secure-bank.xyz

Enter Email Content : Dear Customer,

Click https://bank-login.xyz to verify your account immediately!!

FREE gift awaits.

OUTPUT:

```
==========================================================================
      EMAIL PHISHING DETECTION TOOL
==========================================================================
```

Scanning Keywords...

Suspicious Keyword Found : urgent

Suspicious Keyword Found : verify

Suspicious Keyword Found : bank

Suspicious Keyword Found : account

Suspicious Keyword Found : click

Suspicious Keyword Found : free

Checking URLs...

URLs Found:

https://bank-login.xyz

Checking Sender...

Unknown Sender Domain : secure-bank.xyz

Phishing Score : 17

Classification : HIGH RISK PHISHING EMAIL

Feature Summary

Keyword Analysis : Completed

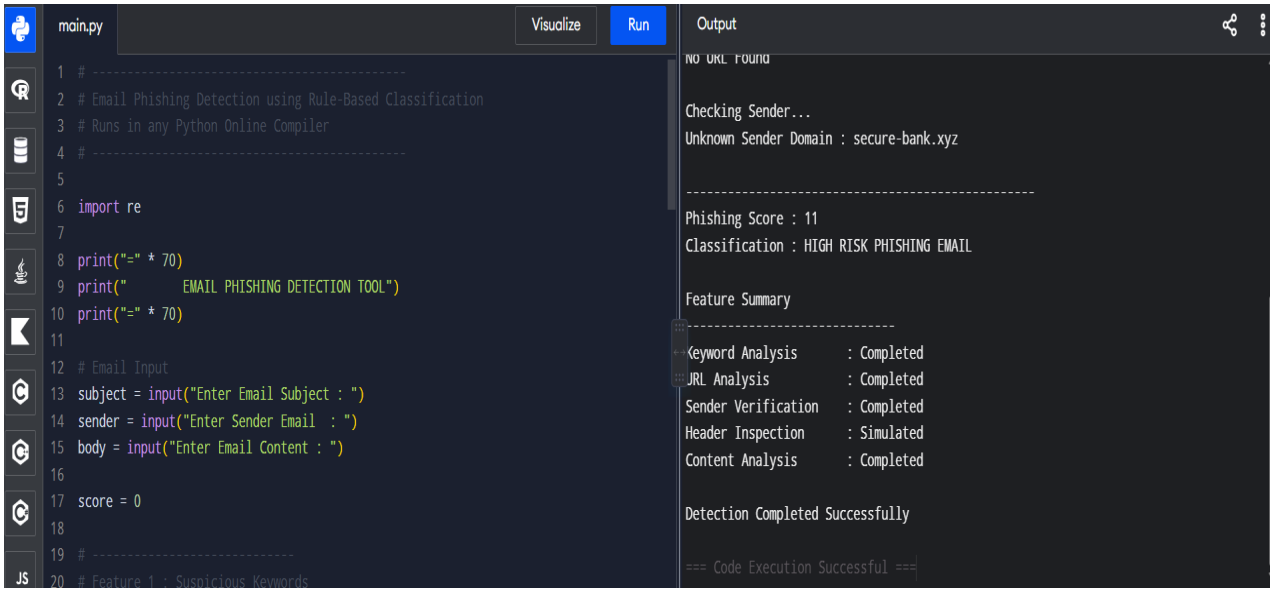
URL Analysis : Completed

Sender Verification : Completed

Header Inspection : Simulated

Content Analysis : Completed

Detection Completed Successfully



Program Explanation

Step 1: Email Input

The user enters:

- Email Subject
- Sender Email Address
- Email Content

Step 2: Feature Extraction

The program extracts several phishing-related features, including:

- Suspicious keywords (e.g., *urgent*, *verify*, *password*, *bank*, *click*)
- URLs present in the email
- Sender domain verification
- Excessive exclamation marks
- Excessive capitalized words

Step 3: Classification

A phishing score is calculated based on the extracted features:

- **0–4:** Safe Email
- **5–7:** Suspicious Email
- **8 or above:** High Risk Phishing Email

Step 4: Result

The program displays the phishing score, classification, and feature analysis summary.

Analysis: Effectiveness of the Model

Strengths

- Detects common phishing indicators such as suspicious keywords and malicious URLs.
- Verifies whether the sender belongs to a trusted domain.
- Fast, lightweight, and suitable for educational demonstrations.
- Does not require external libraries, making it compatible with online Python compilers.

Limitations

- It is a **rule-based classifier**, so it may not detect sophisticated phishing emails that avoid obvious keywords or suspicious domains.
- It does not use machine learning, natural language processing, or real email header analysis.
- Trusted domains can still be spoofed, and legitimate emails may occasionally contain phishing-like keywords, leading to false positives.

Conclusion

This program demonstrates the basic concepts of **email phishing detection** through feature extraction and rule-based classification. In real-world systems, phishing detection is typically enhanced using **machine learning models**, **natural language processing (NLP)**, **URL reputation services**, **SPF/DKIM/DMARC checks**, and **header analysis** to achieve much higher detection accuracy.